

test rules 파이썬 초보자 상세 해설

목표 실제 단위 테스트를 읽으며 파이썬 문법과 프로그램의 판단 흐름을 함께 이해하기

이 문서는 tests/test_rules.py를 대상으로 합니다. 이 파일은 주문 수량 계산 규칙이 여러 상황에서 올바르게 작동하는지 자동으로 검사합니다. 처음부터 모든 수량 규칙을 암기할 필요는 없습니다. 한 테스트를 입력 준비 → 함수 실행 → 결과 확인의 세 덩어리로 나누어 읽는 것이 핵심입니다.

이 파일을 읽는 지도

순서	무엇을 보면 되는가	왜 중요한가
1	import	코드에서 사용할 도구와 데이터 형식을 확인합니다.
2	QuantityRuleTests 클래스	테스트들이 어떤 상자 안에 모여 있는지 이해합니다.
3	decide 보조 함수	공통 기본값과 **문법을 익힙니다.
4	test_로 시작하는 함수	각 상황의 입력값과 기대 결과를 연결합니다.
5	assertEqual 과 assertRaises	정상 결과와 오류 결과를 검증하는 법을 배웁니다.

0 테스트 파일이 하는 일

일반 프로그램은 입력을 받아 결과를 만들지만, 테스트는 반대로 '이 입력이면 이 결과가 나와야 한다'를 미리 적어 둡니다. 프로그램을 수정한 뒤 테스트를 다시 실행하면, 기존 계산 규칙이 우연히 깨지지 않았는지 빠르게 확인할 수 있습니다.

```
■■■■ → decide_quantity ■■ ■■ → ■■ ■■
■■■■ → assertEquals ■ ■■ ■■■■ ■■
```

테스트에서 실패는 나쁜 일이 아닙니다. 실패 메시지는 '어떤 입력에서 기대와 실재가 달랐는가'를 알려 주는 단서입니다.

1 맨 위의 import 문

```
from datetime import datetime
from decimal import Decimal as D
import unittest

from mkr_sync.errors import QuantityRuleError
from mkr_sync.models import BackorderSnapshot, OrderItem, SalesNoteEvidence
from mkr_sync.rules import build_decisions, decide_quantity
```

표현	초보자식 해석
from A import B	A라는 파일 또는 모듈에서 B라는 이름만 가져옵니다.
Decimal as D	Decimal을 이 파일 안에서 D라는 짧은 별칭으로 사용합니다.
import unittest	테스트를 만들고 실행하는 파이썬 표준 도구를 가져옵니다.
from mkr_sync...	이 프로젝트 안의 다른 파일에서 필요한 클래스와 함수를 가져옵니다.

D("100")처럼 따옴표 안에 숫자를 넣는 이유는 Decimal이 정확한 10진수 계산을 하도록 하기 위해서입니다. 주문 수량이나 금액처럼 정확성이 필요한 계산에서는 float보다 Decimal이 안전합니다.

2 클래스와 첫 줄

```
class QuantityRuleTests(unittest.TestCase):
```

class는 관련된 기능을 한 상자에 모으는 문법입니다. QuantityRuleTests는 '수량 규칙 테스트들'이라는 상자 이름입니다. 괄호 안의 unittest.TestCase는 unittest가 제공하는 검사 기능을 물려받는다는 뜻입니다. 그래서

아래에서 self.assertEqual 같은 기능을 사용할 수 있습니다.

3 반복을 줄이는 decide 보조 함수

```
def decide(self, **overrides):
    values = {
        "mkr": "MKR56/26",
        "code": "999999",
        "name": "Test Item",
        "raw_order_qty": D("100"),
        "first_shipped_qty": D("80"),
        "sales_note_order_qty": D("100"),
        "backorder_order_qty": D("0"),
        "backorder_shipped_qty": D("0"),
    }
    values.update(overrides)
    return decide_quantity(**values)
```

이 함수는 거의 모든 테스트에서 공통으로 필요한 기본 주문 데이터를 준비합니다. 각 테스트는 달라지는 값만 전달하면 됩니다. 예를 들어 backorder_order_qty=D("40")을 주면 기본값 0이 40으로 바뀝니다.

표현	의미	예시
def decide	decide라는 함수를 만듭니다.	self.decide(...)
**overrides	이름=값 형태의 추가 인수를 딕셔너리로 받습니다.	code="417133"
values = {...}	키와 값을 짝으로 저장하는 딕셔너리를 만듭니다.	"mkr": "MKR56/26"
update	같은 키가 있으면 기존 값을 새 값으로 바꿉니다.	기본 code를 실제 code로 교체
**values	딕셔너리를 함수 인수 목록으로 펼칩니다.	decide_quantity(mkr=..., code=...)

4 한 테스트를 읽는 기본 공식

```
def test_rule2_already_net(self):
    result = self.decide(
        backorder_order_qty=D("40"),
        backorder_shipped_qty=D("20"),
    )
    self.assertEqual(result.rule_id, "R2_ALREADY_NET")
    self.assertEqual(result.order_qty, D("100"))
```

test_rule2_already_net은 테스트 이름입니다. result = self.decide(...)는 계산을 실행하고 반환 결과를 result 변수에 저장합니다. result.rule_id는 결과 객체 안의 rule_id 속성에 접근하는 표현입니다. assertEquals(실제값, 기대값)은 두 값이 다르면 테스트를 실패시킵니다.

구분	이 코드에서의 모습
준비	backorder_order_qty와 backorder_shipped_qty를 지정합니다.
실행	self.decide(...)가 decide_quantity를 호출합니다.
확인	rule_id와 order_qty가 기대한 값인지 assertEquals로 비교합니다.

5 수량 규칙 테스트 해설

아래 테스트들은 대부분 decide 보조 함수를 사용합니다. 따라서 언급되지 않은 값은 decide 안의 기본값을 그대로 사용합니다. 기본값은 raw_order_qty=100, first_shipped_qty=80, sales_note_order_qty=100, K=0, L=0입니다.

5 1 test_417133_rule3

상황. 원주문 87,336에는 백오더 K 58,536이 포함되어 있고, Sales Note 주문수량은 28,800인 경우입니다.

코드 읽기. self.decide에 code와 네 가지 수량을 직접 전달합니다. 기본값 대신 이 값들이 overrides로 들어갑니다.

무엇을 확인하나. rule_id가 R3_RAW_INCLUDES_K이고 최종 order_qty가 Sales Note의 28,800인지 확인합니다. 즉 원주문에서 백오더를 분리했다는 뜻입니다.

5 2 test_760351_backorder_only_rule1_has_first_precedence

상황. 현재 수량 60과 백오더 출하량 60이 일치해, 실질적으로 백오더만 남아 있는 상황입니다.

코드 읽기. result = self.decide(...) 뒤에 assertEquals의 첫 번째 인수로 세 값을 튜플로 묶습니다.

무엇을 확인하나. R1_BACKORDER_ONLY 규칙과 (0, 0, 0)을 기대합니다. 튜플은 여러 값을 순서대로 한 묶음으로 비교할 때 편리합니다.

5 3 test_760526_rule4

상황. Sales Note의 주문수량 600이 원주문 180과 백오더 출하량 420을 합친 값인 경우입니다.

코드 읽기. D("180")처럼 모든 수량을 Decimal으로 작성합니다. 문자열을 Decimal으로 바꾸는 방식입니다.

무엇을 확인하나. R4_SALES_INCLUDES_L 및 최종 (order, shipped, shortage)가 (180, 120, 60)인지 검사합니다.

5 4 test_760383_rule6_adjusts_shipped

상황. Sales Note 주문수량이 None이고, 백오더 수량 240을 제외해야 하는 상황입니다.

코드 읽기. sales_note_order_qty=None은 숫자 0이 아니라 '자료가 없음'을 의미합니다. assertTrue는 값이 True인지 확인합니다.

무엇을 확인하나. R7_MISSING_SALES_ORDER가 선택되고 shipped_adjusted가 True인지, 결과가 (1800, 1800, 0)인지 확인합니다.

5 5 test_rule2_already_net

상황. 원주문 수량이 이미 현재 주문 기준으로 정리되어 있어 백오더를 다시 빼면 안 되는 상황입니다.

코드 읽기. 이 테스트는 기본값 대부분을 재사용하고 K=40, L=20만 바꿉니다. 보조 함수의 장점이 잘 드러납니다.

무엇을 확인하나. R2_ALREADY_NET과 order_qty 100을 확인합니다. 이름이 규칙의 의도를 설명하는 좋은 예입니다.

5 6 test_rule5_raw_includes_l

상황. 원주문 180과 Sales Note 120의 차이가 백오더 출하량 L 60과 일치하는 상황입니다.

코드 읽기. 한 함수 호출 안에서 여러 키워드 인수를 줄 바꿔 적었습니다. 쉼표는 인수를 구분합니다.

무엇을 확인하나. R5_RAW_INCLUDES_L과 최종 order_qty 120을 확인합니다.

5 7 test_explicit_zero_sales_order_is_not_missing

상황. Sales Note 주문수량이 D("0")인 경우입니다.

코드 읽기. 0은 실제 값이고 None은 값이 없다는 뜻입니다. 이 차이를 테스트 이름과 입력값이 명확히 보여 줍니다.

무엇을 확인하나. 0을 자료 누락으로 오해하지 않고 R3_RAW_INCLUDES_K 및 order_qty 0으로 처리하는지 확인합니다.

5 8 test_sales_mismatch_without_backorder_keeps_original_order

상황. 백오더가 없는데 Sales Note 수량만 원주문과 다른 경우입니다.

코드 읽기. 이름이 긴 테스트도 구조는 같습니다. self.decide 실행 후 rule_id와 order_qty를 비교합니다.

무엇을 확인하나. 백오더가 없으므로 R0_NO_BACKORDER를 선택하고 원주문 100을 유지해야 합니다.

6 오류를 기대하는 테스트

모든 테스트가 정상 결과를 기대하지는 않습니다. 규칙에 맞지 않는 수량은 프로그램이 계산을 멈추고 QuantityRuleError를 내야 합니다. 이 경우 오류 발생이 곧 테스트 성공입니다.

```
with self.assertRaises(QuantityRuleError):
    self.decide(
        sales_note_order_qty=D("70"),
        backorder_order_qty=D("101"),
        backorder_shipped_qty=D("20"),
    )
```

with는 일정 범위 안에서 특별한 동작을 적용하는 문법입니다. 여기서는 들여쓰기된 코드에서 QuantityRuleError가 발생해야 한다는 검사 범위를 만듭니다.

6 1 test_k_greater_than_raw_stops_when_not_rule1

상황. 백오더 주문수량 K가 원주문보다 큰데 R1 조건도 아닌 모순된 상황입니다.

코드 읽기. assertRaises 안에서 self.decide를 실행합니다.

무엇을 확인하나. QuantityRuleError가 나야 합니다. 잘못된 데이터를 조용히 계산하지 않는지 확인합니다.

6 2 test_shipped_above_order_stops

상황. 출하량 101이 주문수량 100보다 큼니다.

코드 읽기. first_shipped_qty만 기본값보다 크게 바꿉니다.

무엇을 확인하나. 출하량이 주문량보다 클 수 없으므로 QuantityRuleError를 기대합니다.

6 3 test_rule6_invalid_shipped_adjustment_stops

상황. Sales Note가 없고 조정 전 출하량이 주문량보다 지나치게 큰 상황입니다.

코드 읽기. 여러 수량을 바꿔 보정 계산이 가능한지 시험합니다.

무엇을 확인하나. 보정 후에도 논리적으로 맞지 않으면 오류를 내야 한다는 안전장치를 검증합니다.

6 4 test_separated_current_order_requires_sales_note_shipped_value

상황. 현재 주문과 과거 백오더가 분리된 품목인데 실제 Sales Note 출하량을 찾지 못한 상황입니다.

코드 읽기. shipped_fallback=True와 backorders_separated=True를 함께 전달합니다.

무엇을 확인하나. 추정 출하량으로 처리하면 안 되므로 QuantityRuleError가 발생해야 합니다.

7 Sales Note가 없거나 백오더가 있는 테스트

이 부분부터는 단순히 decide_quantity만 호출하지 않고 build_decisions도 사용합니다. build_decisions는 여러 주문 품목과 Sales Note 근거, 백오더 목록을 함께 받아 품목별 QuantityDecision 목록을 만듭니다.

```
decisions = build_decisions(
    "MKR49/26",
    {"331383": OrderItem(...)},
    {"331383": SalesNoteEvidence(...)},
    [BackorderSnapshot(...)],
)
result = decisions[0]
```

입력	자료형	역할
{"331383": OrderItem(...)}	dict	품번으로 주문 품목을 찾습니다.
SalesNoteEvidence(...)	객체	Sales Note에서 찾은 주문 및 출하 근거입니다.
[BackorderSnapshot(...)]	list	백오더 기록을 여러 개 담습니다.
decisions[0]	인덱스 접근	결과 리스트의 첫 번째 항목을 꺼냅니다. 파이썬 인덱스는 0부터 시작합니다.

7 1 test_missing_sales_uses_final_order_as_shipped

상황. Sales Note는 없지만 shipped_fallback=True가 주어진 경우입니다.

코드 읽기. None과 bool 값을 함께 사용하고, 반환 결과에서 rule_id와 두 수량을 튜플로 비교합니다.

무엇을 확인하나. R7 규칙에서 최종 주문량 480을 출하량으로도 사용하는 fallback 동작을 검증합니다.

7 2 test_missing_sales_with_k_above_order_uses_l_crosscheck

상황. K가 원주문보다 크지만 L 정보를 사용하면 현재 주문을 계산할 수 있는 경우입니다.

코드 읽기. 긴 테스트 이름은 '어떤 조건에서 어떤 동작을 기대하는가'를 문장처럼 담습니다.

무엇을 확인하나. R8_MISSING_SALES_INCLUDES_L이 선택되고 (300, 300)으로 계산되는지 확인합니다.

7 3 test_separated_current_order_uses_actual_sales_note_without_backorder_deduction

상황. 현재 MKR 주문이 백오더와 이미 분리되어 있고 Sales Note에 실제 출하량이 있는 경우입니다.

코드 읽기. OrderItem 마지막의 True는 backorders_separated 값을 뜻합니다. BackorderSnapshot은 datetime(2026, 9, 1)처럼 날짜와 시간을 받습니다.

무엇을 확인하나. R10을 적용해 현재 주문 1800, 출하 1080, 부족 720을 계산하는지 확인합니다.

7 4 test_mkr49_1_duplicate_code_sales_are_allocated_to_backorders_first

상황. 같은 품번이 여러 과거 백오더에도 있어, Sales Note 출하량을 먼저 과거 백오더에 배정해야 하는 경우입니다.

코드 읽기. BackorderSnapshot 두 개를 리스트에 넣습니다. 리스트는 대괄호로 만들고 항목은 쉼표로 구분합니다.

무엇을 확인하나. 남은 현재 주문의 출하량이 0이고 부족량이 28,800인지 확인합니다.

7 5 test_nonseparated_duplicate_sales_total_can_resolve_backorder_only

상황. 현재 주문과 백오더가 분리되지 않았지만 수량 전체가 백오더와 일치하는 경우입니다.

코드 읽기. decide 보조 함수를 다시 사용합니다. 테스트는 상황에 따라 간단한 함수와 큰 함수를 모두 사용합니다.

무엇을 확인하나. R1_BACKORDER_ONLY로 판단해 현재 주문과 출하량이 모두 0인지 검사합니다.

7 6 test_item_omitted_from_valid_sales_note_is_zero_shipped

상황. 정상적으로 읽힌 Sales Note에 한 품목은 있고 다른 품목은 없는 경우입니다.

코드 읽기. `by_code = {item.code: item for item in decisions}`는 리스트를 품번별 딕셔너리로 바꾸는 딕셔너리 컴프리헨션입니다.

무엇을 확인하나. Sales Note에서 빠진 302509의 `shipped_qty`는 None이 아니라 명시적 0이어야 하고 `fallback`은 False여야 합니다.

8 최신 변경에서 추가된 두 테스트

이 두 테스트는 최신 코드의 중요한 데이터 의미를 보여 줍니다. 핵심은 값이 0인 것, 아직 알 수 없는 것, 주문서에는 없지만 Sales Note에는 있는 것을 서로 구분하는 것입니다.

8 1 test_missing_sales_note_keeps_shipment_and_shortage_unknown

상황. 주문 품목은 있지만 유효한 Sales Note가 전혀 없는 경우입니다.

코드 읽기. `build_decisions`에 `order_items` 딕셔너리, 빈 `sales_evidence` 딕셔너리 {}, 빈 백오더 리스트 [], `sales_note_available=False`를 전달합니다. `OrderItem`의 `origin="Japan"`은 키워드 인수입니다.

무엇을 확인하나. `order_qty`는 72로 알 수 있지만 `shipped_qty`와 `shortage_qty`는 None이어야 합니다. `record_status`는 `PENDING_SALES_NOTE`입니다. None은 '아직 모름'이라는 정보입니다.

8 2 test_sales_note_only_item_is_kept_after_backorder_allocation

상황. 주문서에는 없지만 Sales Note에는 품목 162365가 있는 경우입니다.

코드 읽기. `order_items`는 빈 딕셔너리이고, `sales_evidence`에는 `SalesNoteEvidence` 하나가 있습니다. `sales_note_source="latest.pdf"`처럼 출처도 전달합니다.

무엇을 확인하나. `R12_SALES_NOTE_ONLY` 규칙을 사용하고 주문량과 출하량 72를 보존하며 `record_status`를 `SALES_NOTE_ONLY`로 표시하는지 확인합니다.

9 자주 보이는 파이썬 문법 요약

문법	이 파일에서의 예	해석
문자열 str	"MKR56/26"	글자 데이터를 큰따옴표 또는 작은따옴표로 감쌉니다.
정확한 소수 Decimal	D("100")	수량 계산에서 오차를 피하기 위한 10진수입니다.
None	<code>sales_note_order_qty=None</code>	0이 아니라 값이 아직 없거나 알 수 없다는 뜻입니다.
bool	True, False	조건의 참과 거짓을 나타냅니다.
dict	<code>{"331383": OrderItem(...)}</code>	키로 값을 찾는 자료형입니다.
list	<code>[BackorderSnapshot(...)]</code>	순서가 있는 여러 값을 담습니다.
tuple	<code>(order, shipped, shortage)</code>	여러 값을 순서대로 묶어 한 번에 비교합니다.
속성 접근	<code>result.order_qty</code>	객체 안에 저장된 정보를 꺼냅니다.
예외	<code>assertRaises(QuantityRuleError)</code>	잘못된 입력을 오류로 막는지 검사합니다.

10 이 파일을 혼자 읽는 방법

1. 테스트 이름을 한국어 문장으로 바꿔 말합니다. 2. `self.decide` 또는 `build_decisions`의 괄호 안에서 바뀐 입력값만 찾습니다. 3. `result` 또는 `decisions`에 어떤 결과가 담겼는지 확인합니다. 4. `assertEqual`, `assertTrue`, `assertFalse`, `assertIsNone`, `assertRaises`가 무엇을 기대하는지 읽습니다. 5. 기대값이 왜 필요한지 함수 `rules.py`에서 다시 확인합니다.